Лабораторная работа 2

ОРГАНИЗАЦИЯ ЦИФРОВОГО ВВОДА-ВЫВОДА

Цель работы

Изучение основ работы с цифровыми портами ввода-вывода микроконтроллера ATmega32. Получение практических навыков по обработке внешних прерываний и организации ввода-вывода с помощью механизма прерываний.

Теоретические сведения

Порты ввода-вывода

Связь микроконтроллера с внешними устройствами осуществляется через порты ввода-вывода. Каждый разряд порта соответствует одному из выводов микросхемы. В зависимости от модели микроконтроллера может быть от одного до семи восьмиразрядных портов (иногда присутствуют неполные порты, имеющие менее восьми разрядов), именующихся латинскими буквами от A до G. Упрощённая схема одного вывода представлена на рисунке 3. Физическое расположение выводов можно найти в спецификации микроконтроллера.

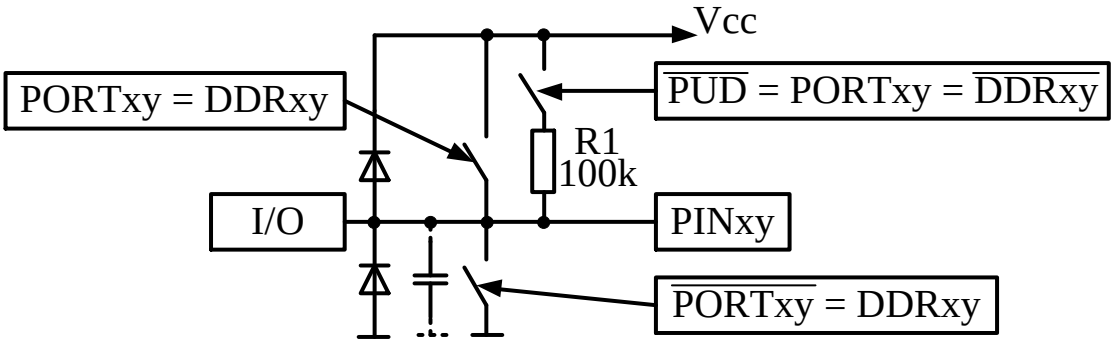


Рис. 1. Схема порта ввода-вывода

Управляется каждый порт тремя специальными регистрами – PORTx, DDRx и PINx, где под «x» подразумевается конкретная буква – имя порта. Отдельные разряды регистров также имеют свои имена. Разряды регистра PORTx именуются Pxi, где «i» – номер соответствующего разряда. Например, разряды порта A именуются PA0, PA1, …, PA7. Аналогично,

разряды порта DDRx именуются DDxi (DDA0, ..., DDA7 для порта A), а разряды регистра PINx именуются PINxi (PINA0, ..., PINA7).

Каждый контакт микросхемы устроен так, что может работать как на ввод, так и на вывод информации, а для переключения режима (ввод/вывод) служит регистр DDRx. Каждый разряд этого регистра управляет соответствующим разрядом порта – если разряд равен нулю, то разряд порта работает как вход. Если разряд регистра равен единице, то разряд порта работает как выход. Таким образом, все выводы микроконтроллера настраиваются независимо друг от друга.

Каждый вывод имеет в своём составе диод, защищающий от превышения напряжения. Изображённый пунктиром конденсатор обозначает паразитарную ёмкость вывода – несмотря на её маленькую величину, она всё же присутствует, и иногда её приходится учитывать. Всё управление выводом осуществляется ключами управления через регистры порта. Для простоты изложения на схеме ключи обозначены в виде переключателей, хотя в реальности это полевые транзисторы.

Чтобы выдать нужный логический уровень на вывод микросхемы, нужно записать в соответствующий разряд регистра DDRx единицу, а затем записать бит данных в соответствующий разряд регистра PORTx. Логический уровень будет присутствовать на выводе порта до тех пор, пока линия не переключится на ввод, либо не изменится значение регистра PORTx.

Для того чтобы прочитать информацию с внешнего вывода микроконтроллера, необходимо сначала перевести нужный разряд порта в режим ввода. То есть записать в соответствующий разряд регистра DDRx ноль. Только после этого на данный вывод микроконтроллера можно подавать цифровой сигнал от внешнего устройства. Далее микроконтроллер просто читает байт из регистра PINx. Содержимое соответствующего бита прочитанного байта будет соответствовать сигналу на внешнем выводе порта.

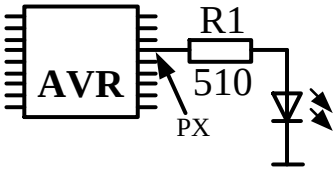
Каждый вывод дополнительно оснащён подтягивающим (pull-up) резистором. Подтягивающие резисторы не имеют независимого управления и могут быть подключены либо отключены одновременно установкой/сбросом бита PUD (PullUp Disable) регистра SFIOR. Логика управления является инверсной, т.е. установка бита отключает резисторы. По умолчанию бит равен нулю, и подтягивающие резисторы включены.

Пример подключения внешних устройств

Исследовать работу портов ввода-вывода можно при помощи светодиодов и контактных кнопок, подключённых к выводам микроконтроллера. Напишем программу, которая будет при нажатии на подключённую к микроконтроллеру кнопку зажигать/гасить светодиод.

Для начала разберём схему подключения. Поскольку логической единице соответствует напряжение 5 В, а рабочее напряжение светодиодов составляет около 3 В, то подключать светодиод напрямую нельзя – это приведёт к его сгоранию. Для правильного подключения нужно использовать сопротивление. Подключить светодиод можно двумя способами.

Зажигание логической единицы



Зажигание логического нуля

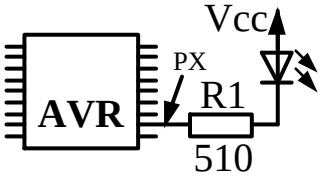


Рис. 2. Различные способы использования порта

В первом случае для включения светодиода нужно сконфигурировать порт на вывод, и записать в него единицу. Во втором случае нужно сконфигурировать порт на ввод, и записать в него ноль. Мы воспользуемся первым способом, как более прозрачным с точки зрения логики.

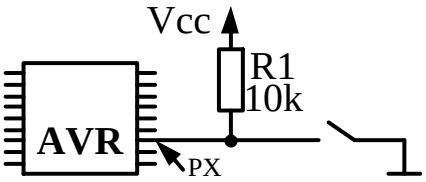


Рис. 3. Вариант подключения кнопки

Разберём чуть подробнее схему подключения кнопки. Для корректной работы кнопки регистр ввода-вывода должен быть сконфигурирован на ввод. На рисунке 5 изображён подтягивающий резистор *R1*. Подтягивающие резисторы уже встроены в микроконтроллер, и на схеме резистор *R1* приведён только для наглядности изложения. Видно, что когда кнопка отпущена, с входа микроконтроллера будет считываться логическая единица, т. к. вход подтянут резистором к линии питания. Когда кнопка нажата, то линия питания соединяется с землёй через резистор, при этом из регистра PINx будет считан логический ноль.

Подсистема прерываний

Все микроконтроллеры AVR имеют многоуровневую систему прерываний. Прерывание прекращает нормальный ход программы для выполнения приоритетной задачи, определяемой внутренним или внешним событием.

Для каждого такого события разрабатывается отдельная программа, которую называют подпрограммой обработки запроса на прерывание (для краткости – подпрограммой прерывания), и размещается в памяти программ.

У каждого периферийного устройства, что входит в состав AVR микроконтроллеров, есть как минимум один источник прерывания (Interrupt source). Ко всем этим прерываниям следует причислить и прерывание сброса – Reset Interrupt, предназначение которого отличается от всех остальных.

За каждым прерыванием, строго закреплён вектор (ссылка) указывающий на процедуру обработки прерывания (Interrupt service routine). Все векторы прерываний, располагаются в самом начале памяти программ и вместе формируют «таблицу векторов прерываний» (Interrupt vectors table).

Каждому прерыванию соответствует определённый «бит активации прерывания» (Interrupt Enable bit). Таким образом, чтобы использовать определённое прерывание, следует записать в его «бит активации прерывания» – логическую единицу. Далее, независимо от того активированы ли определённые прерывания, микроконтроллер не начнёт обработку этих прерываний, пока в «бит всеобщего разрешения прерываний» (Global Interrupt Enable bit в регистре состояния *SREG*) не будет записана логическая единица. Также, чтобы запретить все прерывания (на неопределённое время), в бит всеобщего разрешения прерываний следует записать – логический ноль.

В таблице 1 приведён перечень источников прерываний для МК ATmega32.

Таблица1 — Источники прерываний в МК ATmega32

№	Адрес	Источник	Описание
1	0x000	RESET	Сигнал сброса
2	0x002	INT0	Внешний запрос на прерывание по входу INT0

3	0x004	INT1	Внешний запрос на прерывание по входу INT1
4	0x006	INT2	Внешний запрос на прерывание по входу INT2
5	0x008	TIMER2_COMP	Совпадение с регистром сравнения таймера T/C2
6	0x00A	TIMER2_OVF	Переполнение счётчика T/C2
7	0x00C	TIMER1_CAPT	Захват по таймеру T/C1
8	0x00E	TIMER1_COMPA	Совпадение с регистром сравнения A таймера T/C1
9	0x010	TIMER1_COMPB	Совпадение с регистром сравнения B таймера T/C1
10	0x012	TIMER1_OVF	Переполнение счётчика T/C1
11	0x014	TIMER0_COMP	Совпадение с регистром сравнения таймера T/C0
12	0x016	TIMER0_OVF	Переполнение счётчика T/C0
13	0x018	SPI_STC	Передача данных по интерфейсу SPI завершена
14	0x01A	UART_RXC	Приём данных приёмопередатчиком UART завершён
15	0x01C	UART_UDRE	Регистр данных UART пуст
16	0x01E	UART_TXC	Передача данных приёмопередатчиком UART завершена
17	0x020	ADC	Завершено преобразование АЦП
18	0x022	EE_RDY	EEPROM готов
19	0x024	ANA_COMP	Прерывание от аналогового компаратора
20	0x026	TWI	Прерывание от интерфейса I2C
21	0x028	SPM_RDY	Запись программной памяти (Flash) готова

Управление внешними прерываниями

За управление внешними прерываниями в ATmega32 отвечают четыре регистра:

– GICR (он же GIMSK) – запрет/разрешение прерываний по сигналам на входах INT0, INT1;

– MCUCR – выбор условия срабатывания прерываний int0 и int1;

– GIFR – управление внешними прерываниями;

7	6	5	4	3	2	1	0
INT1	INT0	-	-	-	-	-	-

Рис. 4. Регистр GICR

INTx=1: прерывания по сигналу на входы INTx разрешены.

7	6	5	4	3	2	1	0
SE	SM2	SM1	SM0	ISC11	ISC10	ISC01	ISC00

Рис. 5. Регистр MCUCR

ISC01 и ISC00 – управление прерыванием int0:

ISC01	ISC00	Описание
0	0	Низкий уровень напряжение вызывает прерывание
0	1	Любое изменение логического сигнала вызывает прерывание
1	0	Перепад 1/0 вызывает прерывание
1	1	Перепад 0/1 вызывает прерывание

Аналогично биты ISC11 и ISC10 управляют прерыванием int1.

7	6	5	4	3	2	1	0
INTF1	INTF0	-	-	-	-	-	-

Рис. 6. Регистр GIFR

INTFx=1: произошло прерывание на входе INTx. При входе в подпрограмму обработки прерывания INTFx автоматически сбрасывается в состояние логического нуля.

При входе в подпрограмму обработки прерывания соответствующий прерыванию флаг регистра GIFR автоматически сбрасывается в состояние логический ноль.

Прерывания работают только тогда, когда в регистре состояния SREG разрешены общие прерывания (бит 7 равен единице). В случае наступления прерывания этот бит автоматически сбрасывается в ноль, блокируя выполнение последующих прерываний.

Примеры решения задач

Считывание нескольких нажатых кнопок

Учитывая высокую частоту работу микроконтроллера, при вводе многоразрядного числа с помощью кнопок, подключённых к отдельным разрядам портов ввода-вывода, необходимо обеспечить временную задержку между зажатием первой кнопки и вводом необходимой комбинации кнопок. Кроме этого, необходимо обеспечить защиту от повторного выполнения фрагмента кода, обеспечивающего ввод числа, – проще всего установить цикл, ожидающий пока все кнопки не будут отпущены, перед выходом из фрагмента кода.

Рис. 7. Блок-схема считывания многоразрядного числа

Пример кода на ассемблере:

```
.def NULL = R16
.def TMP  = R17

read_number:
   IN   TMP, PINB
   CP   TMP, NULL ; PINB = 0?
   BREQ read_number; если PINB = 0 возврат в начало функции
   CALL delay ; ожидание нажатия комбинации кнопок
   IN   TMP, PINB ; чтение комбинации кнопок
   CALL operation ; выполнение операции
stop_reading: ; обеспечение однократного ввода
   IN   TMP, PINB
   CP   TMP, NULL ; PINB = 0?
   BRNE stop_reading ; ожиданіе условія PINB == 0
   RET
```

Пример кода на си:

```
uint_8t get_num() {
  uint_8t Num;
  while(!PINA); // ожидания нажатия каких-либо кнопок на порту A
  delay_ms(120);// пауза для завершения ввода сочетания кнопок
  Num = PINA;   // считывание сочетания кнопок
// обработка введённого числа
  while(PINA);  // ожидание завершения ввода (обеспечение однократной
обработки введённого значения)
}
```

Пример использования обработчика прерывания int0

```
.org $000
   JMP RESET            ; Указатель на начало программы
.org INT0addr
   JMP EXT_INT0   ; Указатель на обработчик прерывания int0
.org INT1addr
   JMP EXT_INT1   ; Указатель на обработчик прерывания int1

RESET:            ; Начало основной программы
      LDI R20, HIGH(RAMEND) ; Старшие разряды адреса
      OUT SPH, R20            ; Установка вершины стека в конец ОЗУ
      LDI R20, LOW(RAMEND)  ; Младшие разряды адреса
      OUT SPL, R20            ; Установка вершины стека в конец ОЗУ
      SER R16
      OUT DDRA, R20           ; Настройка PORTA на вывод
      OUT DDRB, R20           ; Настройка PORTB на вывод
      LDI R16, 0x0F
      OUT MCUCR, R16    ; Настройка прерываний int0 и int1 на условие 0/1
      LDI R16, 0xC0
      OUT GICR, R16           ; Разрешение прерываний int0 и int1
      OUT GIFR, R16           ; Предотвращение срабатывания int0 и int1 при
включении прерываний
      SEI               ; Включение прерываний

LOOP:
      NOP
      ; ...
      ; здесь могут быть операции с регистром R16, например:
      ; DEC R16 ; <- если поступит прерывание и в обработчике будет перезаписан
R16 (1)
      ; а также условные переходы, например:
      ; CPI R16, 0xef ; <- если поступит прерывание и в обработчике будет
перезаписан SREG (2)
      ; BRNE LABEL1 ; <- при выполнении условий (1) или (2) переход (не) будет
выполнен ошибочно
      ; ...
      RJMP LOOP   ; Бесконечный цикл

EXT_INT0:         ; Обработчик прерывания int0
      PUSH R16    ; Сохранение текущего значения R16 в стеке
      IN R16, SREG
      PUSH R16          ; Сохранение текущего значения SREG в стеке
      IN R16, PORTA           ; (!) изменяется значение R16
      COM R16           ; (!!!) изменяются значения флагов в SREG
      OUT PORTA, R16    ; Инверсия значения на PORTA
      POP R16
      OUT SREG, R16           ; Восстановление значения SREG из стека
      POP R16           ; Восстановление значения R16 из стека
      RETI              ; Возврат из обработчика прерываний и разрешение
прерываний

EXT_INT1:               ; Обработчик прерывания int1
      PUSH R16          ; Сохранение текущего значения R16 в стеке
      IN R16, SREG
      PUSH R16          ; Сохранение текущего значения SREG в стеке
      IN R16, PORTB           ; (!) изменяется значение R16
```

```
COM R16            ; (!!!) изменяются значения флагов в SREG
OUT PORTB, R16     ; Инверсия значения на PORTB
POP R16
OUT SREG, R16            ; Восстановление значения SREG из стека
POP R16            ; Восстановление значения R16 из стека
RETI               ;  Возврат  из  обработчика  прерываний  и  разрешение
прерываний
```

Пример алгоритма для калькулятора

Задача: реализовать простейший восьмиразрядный калькулятор. Регистр PORTA отображает результат предыдущей операции или первый введённый операнд. Регистр PORTB используется для ввода восьмиразрядного знакового операнда в двоичной системе счисления (необходима временная задержка, позволяющая зажать несколько кнопок одновременно). Разряды регистра PORTC отвечают за указание математической операции (PC1 – логическое сложение, PC2 – арифметическое вычитание) или обнуление первого операнда (PORTA) – PC0 и вывод флагов (переполнение – PC7 и отрицательный результат – PC6). Ввод данных должен осуществляться в следующей последовательности: ввод первого операнда, указание действия, ввод второго операнда. После выполнения первой операции в качестве первого операнда может выступать результат предыдущей операции. Ввод второго операнда до указания действия приводит к замене первого операнда. Повторное указание действия перед вводом второго операнда приводит к замене выполняемого действия.

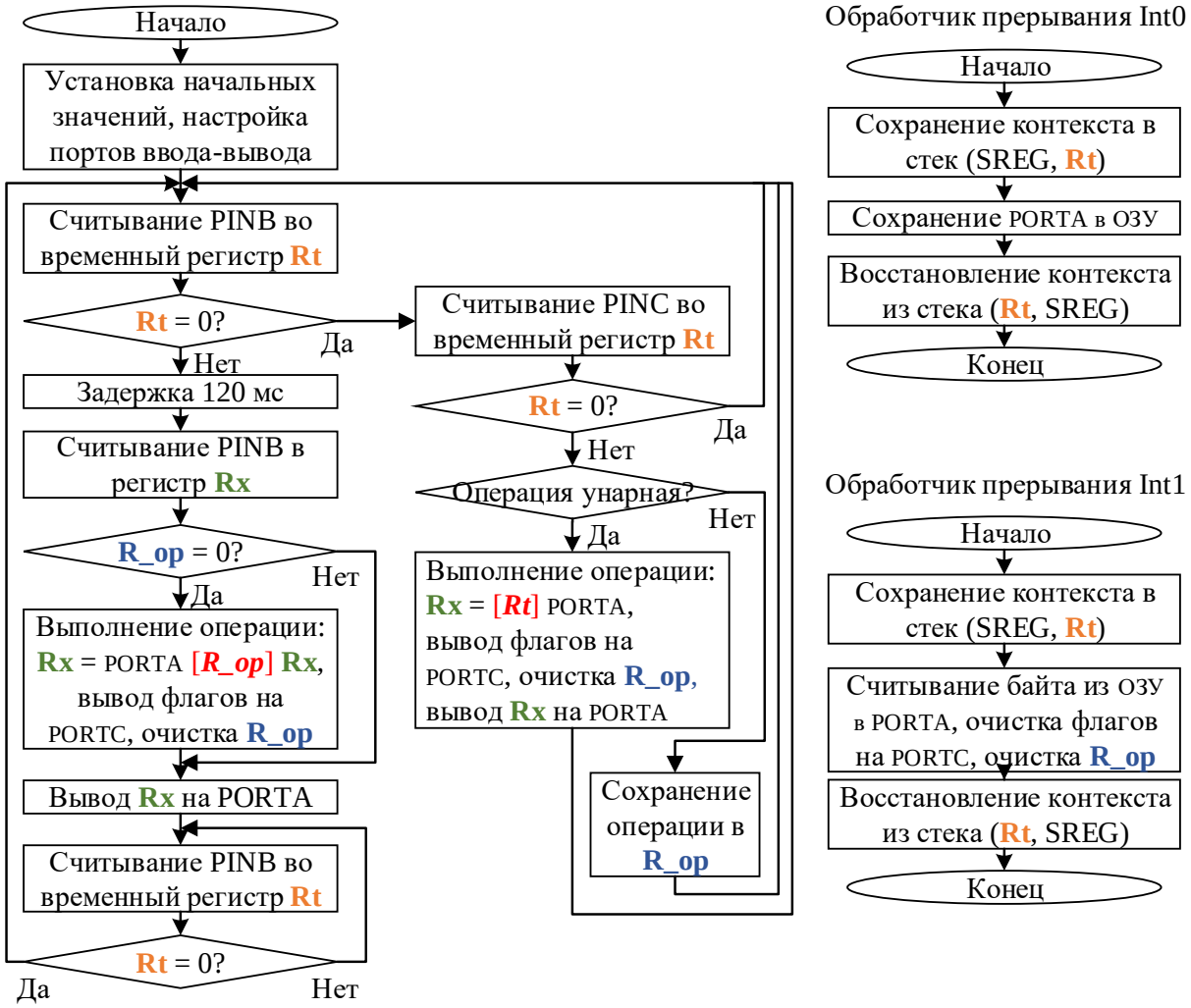


Рис. 8. Блок-схема алгоритма работы калькулятора

```
.def NULL = R16
.def TMP  = R17
.def TMP2 = R18
.def OP   = R19
.def FLAG = R20

reset:
; настройка исходных значений
   CLR  NULL ; 0x00
   CLR  OP   ; 0x00
; настройка портов ввода-вывода
   SER  TMP ; 0xFF
   OUT  DDRA, TMP ; Вывод
   CLR  TMP ; 0x00
   OUT  DDRB, TMP ; Ввод
   LDI  TMP, 0xF8
   OUT  DDRC, TMP ; 7,6 - вывод, 2,1,0 - ввод
; Установка вершины стека в конец ОЗУ
   LDI  TMP, HIGH(RAMEND) ; Старшие разряды адреса
   OUT  SPH, TMP
   LDI  TMP, LOW(RAMEND) ; Младшие разряды адреса
   OUT  SPL, TMP

read_number: ; считывание операнда
   IN   TMP, PINB
   CP   TMP, NULL ; PINB = 0?
   BREQ read_operation ; если PINB = 0 считывание операции
   CALL delay ; ожидание нажатия комбинации кнопок
   IN   TMP, PINB ; чтение комбинации кнопок
   CPSE OP, NULL ; если введена операция
   CALL operation ; выполнение операции
   OUT  PORTA, TMP ; вывод операнда или результата операции
stop_reading: ; обеспечение однократного ввода
   IN   TMP, PINB
   CP   TMP, NULL ; PINB = 0?
   BRNE stop_reading ; ожидание условия PINB == 0
   RJMP read_operation

read_operation: ; считывание операции
   IN   TMP, PINC
   CP   TMP, NULL ; PINC = 0?
   BREQ read_number ; если PINC = 0 считывание операнда
   SBRC TMP, 0 ; если операция не равна очистке: PC.0 – очистка
   RJMP op_null ; выполнение операции очистки
   MOV  OP, TMP ; сохранение операции
   RJMP read_number

operation: ; начало бинарной операции
   IN   TMP2, PORTA ; считывание первого операнда из PORTA
   SBRC OP, 1 ; определение операции: PC.1 – OR, PC.2 – SUB
   RJMP op_or ; если PC1 = 1 выполнение операции OR
   RJMP op_sub ; если PC1 = 0 выполнение операции SUB

op_null: ; операция очистки
   OUT  PORTA, NULL ; очистка операнда
   MOV  FLAG, NULL ; очистка флагов
   CALL end_of_operation
   RJMP read_number

op_or: ; операция OR
   OR   TMP, TMP2 ; операция OR
   IN   FLAG, SREG ; сохранение флагов
   RJMP end_of_operation

op_sub: ; операция SUB
   SUB  TMP2, TMP ; операция SUB
   MOV  TMP, TMP2 ; перенос результата
   IN   FLAG, SREG ; сохранение флагов
   RJMP end_of_operation
```

```
end_of_operation: ; завершение операции
   SWAP FLAG ; замена тетрад: PC.7 = SREG.3 (V), PC.6 = SREG.2 (N)
   ANDI FLAG, 0xC0 ; очистка других флагов
   OUT  PORTC, FLAG ; вывод флагов
   MOV  OP, NULL ; очистка операции
   RET

delay: ; задержка 120 мс
   LDI  R31, 5
   LDI  R30, 223
   LDI  R29, 188
delay_sub:
   DEC  R29
   BRNE delay_sub
   DEC  R30
   BRNE delay_sub
   DEC  R31
   BRNE delay_sub
   NOP
   NOP
   RET
```

Представление чисел со знаком

1. Прямой код:

Старший бит кодирует знак

Остальные биты совпадают у положительного и отрицательного числа

2. Обратный код (дополнение до 1):

Старший бит кодирует знак

a + (-a) = 2n-1, где n – количество разрядов числа

3. Дополнительный код (дополнение до 2):

Старший бит кодирует знак

a + (-a) = 2n, где n – количество разрядов числа

Примеры:

Число		Прямой код	Обратный код	Дополнительный код
23	10111	00010111	00010111	00010111
127	1111111	01111111	01111111	01111111
1	1	00000001	00000001	00000001
-1	-1	10000001	11111110	11111111
-17	-10001	10010001	11101110	11101111
-127	-1111111	11111111	10000000	10000001

Содержание отчёта

1. Схема установки (задействованные узлы отладочной платы).
2. Блок-схема алгоритма работы программы.
3. Временная диаграмма цифровых сигналов на портах ввода-вывода (фрагмент).
4. Комментированный листинг программы на языке ассемблера.

5. Ответы на контрольные вопросы.

Контрольные вопросы

1. Какими способами можно подключить внешние устройства (светодиод, кнопку) к микроконтроллеру?
2. Как реализуется подсистема прерываний в микроконтроллере AVR?
3. Как программно разрешить или запретить выполнение конкретного прерывания?
4. Какие источники прерываний есть в микроконтроллерах AVR?
5. Как настраиваются внешние прерывания?

Варианты заданий

В описании вариантов символ (*) обозначает, что число ***-y*** вычисляется и представляется в прямом, обратном и дополнительном коде в зависимости от номера студента в составе бригады:

Номер варианта	Правило вычисления ***-y***
Ха	Прямой код
Хб	Обратный код
Хв	Дополнительный код

Варианты логики работы внешних прерываний INT0 и INT1, светодиодов и кнопок на портах PORTA, PORTB, PORTC и PORTD:

Номер варианта	Задание
1	Регистры PORTA–PORTB выполняют роль новогодней гирлянды. Есть следующие 3 режима работы: – на каждом регистре два состояния – вывод чисел *0xFF* и *0x00*, смена состояний с частотой ***x***; – на каждом регистре два состояния – вывод чисел *0xAA* и *0x55*, смена состояний с частотой ***x***; – на каждом регистре два состояния – вывод чисел ***y*** и ***-y****, смена состояний с частотой ***x***. Регистры PD4-PD5 отображают номер режима работы (1-3), регистр PD6 отображают номер состояния в конкретном режиме (0/1). Ввод числа ***y*** должен выполняться на PORTC (одновременным нажатием на несколько кнопок) при нажатой кнопке PD7. Нажатие кнопки PD7 может (а вернее должно) приводить к остановке

	основного функционала (вывод информации на светодиоды) до момента отпускания всех кнопок на PORTC или отпускания PD7, после чего основной функционал возобновиться с того же режима и состояния, но с новым значением ***y***. Смена режима работы должна производиться циклически с помощью кнопок PD2 (прерывание INT0) и PD3 (прерывание INT1), в прямом и обратном порядке соответственно, при этом номер активного состояния должен сохраняться. Изменения должны отображаться на регистре PORTD и сохраняться во внутренний EEPROM МК в момент нажатия на кнопки PD2 и PD3. Исходное положение: первое состояние, величина ***x*** = *2* Гц, ***y*** = 0xAA, номер режима извлекается из EEPROM, место и формат хранения выбирается самостоятельно.
2	Разрядам регистров PORTA и PORTB ставится в соответствие отрезок [0–15] (0-7 ~ PA0–PA7, 8-15 ~ PB0-PB7). На данном отрезке отображается «бегущий огонь» из нескольких горящих светодиодов (восьмиразрядное число ***y*** и ***-y****, например, если *y*=53=*0x*35=*0b***0011.0101**, то в варианте с обратным кодом «бегущий огонь» выглядит как *0b***0011.0101.1100.1010**). Для каждого состояния активные светодиоды загорается на ***z*** с, после чего активным становится другие светодиоды. Номер следующего активного светодиода определяется по формуле: $n_{i+1} = n_i + x \ (mod\ 16)$, где n_i – номер текущего светодиода, n_{i+1} – номер следующего светодиода, ***x*** – величина шага, лежащая в интервале [-3;+3]. Регистры PD0-PD1 отображают номер элемента ***z*** (1-3) в множестве величин задержки {0,25 с; 0,5 с; 1 с}; PD4-PD6 отображают значение ***x*** в прямом коде. Ввод числа ***y*** должен выполняться на PORTC (одновременным нажатием на несколько кнопок) при нажатой кнопке PD7. Нажатие кнопки PD7 может (а вернее должно) приводить к остановке основного функционала (вывод информации на светодиоды) до момента отпускания всех кнопок на PORTC или отпускания PD7, после чего основной функционал возобновиться с того же режима и состояния, но с новым значением ***y***. Кнопки PD2 (прерывание INT0) увеличивает величину шага ***x*** на единицу за одно нажатие до +3, после чего устанавливает её равной -3 (одно нажатие – одно изменение).

	Кнопка PD3 (прерывание INT1) циклически изменяет время задержки ***z*** в множестве величин задержки {0,25 с; 0,5 с; 1 с}. Изменение величин шага и задержки должно отображаться на регистре PORTD и сохраняться во внутренний EEPROM МК в момент нажатия на кнопки PD2 и PD3, изменение логики отображения – со следующего шага «бегущего огня». Исходное положение: величина шага ***x*** = *1*, величина задержки ***z*** извлекается из EEPROM, место и формат хранения выбирается самостоятельно, PORTA = ***y*** = *0x83*.
3	Регистры PORTA–PORTB выполняют роль новогодней гирлянды. Есть следующие 2 режима работы: – на каждом регистре два состояния – вывод чисел *0xFF* и *0x00*, смена состояний с частотой ***x***; – на каждом регистре два состояния – вывод чисел ***y*** и ***-y****, смена состояний с частотой ***x***. Регистр PD4 отображают номер режима работы (0/1), регистр PD5 отображают номер состояния в конкретном режиме (0/1), регистры PD0-PD1 отображают номер элемента ***x*** в множестве частот. Ввод числа ***y*** должен выполняться на PORTC (одновременным нажатием на несколько кнопок) при нажатой кнопке PD7. Нажатие кнопки PD7 может (а вернее должно) приводить к остановке основного функционала (вывод информации на светодиоды) до момента отпускания всех кнопок на PORTC или отпускания PD7, после чего основной функционал возобновиться с того же режима и состояния, но с новым значением ***y***. Смена режима работы должна производиться циклически с помощью кнопки PD2 (прерывание INT0), при этом номер активного состояния должен сохраняться. С помощью кнопки PD3 (прерывание INT1) циклически изменяется величина ***x*** в следующем множестве {0,25 Гц; 0,5 Гц; 1 Гц}. Изменения должны отображаться на регистре PORTD и сохраняться во внутренний EEPROM МК в момент нажатия на кнопки PD2 и PD3. Исходное положение первый режим и первое состояние (0xFF на регистрах PORTA-PORTC), величина ***x*** извлекается из EEPROM, место и формат хранения выбирается самостоятельно, ***y*** = 0x55.

4	Разрядам регистров PORTA и PORTB ставится в соответствие отрезок [0–15] (0-7 ~ PA0–PA7, 8-15 ~ PB0-PB7). На данном отрезке отображается «бегущий огонь» из нескольких горящих светодиодов (восьмиразрядное число ***y*** и ***-y****, например, если *y*=53=*0x*35=*0b***0011.0101**, то в варианте с обратным кодом «бегущий огонь» выглядит как *0b***0011.0101.1100.1010**). Для каждого состояния активные светодиоды загорается на 0,2 с, после чего активным становится другие светодиоды. Номер следующего активного светодиода определяется по формуле: $n_{i+1} = n_i + x \ (mod\ 16)$, где n_i – номер текущего светодиода, n_{i+1} – номер следующего светодиода, ***x*** – величина шага, лежащая в интервале [-3;+3]. Разряды PD4-PD6 отображают значение ***x*** в прямом коде. Ввод числа ***y*** должен выполняться на PORTC (одновременным нажатием на несколько кнопок) при нажатой кнопке PD7. Нажатие кнопки PD7 может (а вернее должно) приводить к остановке основного функционала (вывод информации на светодиоды) до момента отпускания всех кнопок на PORTC или отпускания PD7, после чего основной функционал возобновиться с того же режима и состояния, но с новым значением ***y***. Кнопка PD2 (прерывание INT0) увеличивает величину шага ***x*** до +3, после чего устанавливает её равной -3 (одно нажатие – одно изменение). Кнопка PD3 (прерывание INT1) меняет взаимное расположение ***y*** и ***-y**** в «бегущем огне». Изменения величины шага должны отображаться на регистре PORTD и сохраняться во внутренний EEPROM МК в момент нажатия на кнопки PD2 и PD3, изменение логики отображения – со следующего шага «бегущего огня». Исходное положение: PORTA = ***y*** = *0x83*, величина шага ***x*** извлекается из EEPROM, место и формат хранения выбирается самостоятельно.
5	Регистры PORTA–PORTB выполняют роль новогодней гирлянды. Есть следующие 3 режима работы: – на каждом регистре два состояния – вывод чисел *0xFF* и *0x00*, смена состояний с частотой ***x***; – на каждом регистре два состояния – вывод чисел *0xAA* и *0x55*, смена состояний с частотой ***x***; – на каждом регистре два состояния – вывод чисел ***y*** и ***-y****, смена состояний с частотой ***x***.

	Регистры PD0-PD1 отображают номер режима работы (1-3), регистр PD4 отображают номер состояния в конкретном режиме (0/1). Ввод числа ***y*** должен выполняться на PORTC (одновременным нажатием на несколько кнопок) при нажатой кнопке PD7. Нажатие кнопки PD7 может (а вернее должно) приводить к остановке основного функционала (вывод информации на светодиоды) до момента отпускания всех кнопок на PORTC или отпускания PD7, после чего основной функционал возобновиться с того же режима и состояния, но с новым значением ***y***. Смена режима работы должна производиться циклически с помощью кнопок PD2 (прерывание INT0) и PD3 (прерывание INT1), в прямом и обратном порядке соответственно, при этом номер активного состояния должен сохраняться. Изменения должны отображаться на регистре PORTD и сохраняться во внутренний EEPROM МК в момент нажатия на кнопки PD2 и PD3. Исходное положение: первое состояние и величина ***x*** = *2* Гц, ***y*** = 0x55, номер режима извлекается из EEPROM, место и формат хранения выбирается самостоятельно.
6	Разрядам регистров PORTA и PORTB ставится в соответствие отрезок [0–15] (0-7 ~ PA0–PA7, 8-15 ~ PB0-PB7). На данном отрезке отображается «бегущий огонь» из нескольких горящих светодиодов (восьмиразрядное число ***y*** и ***-y****, например, если y=53=*0x*35=*0b***0011.0101**, то в варианте с обратным кодом «бегущий огонь» выглядит как *0b***0011.0101.1100.1010**). Для каждого состояния активные светодиоды загорается на ***z*** с, после чего активным становится другие светодиоды. Номер следующего активного светодиода определяется по формуле: $n_{i+1} = n_i + x \ (mod\ 16)$, где n_i – номер текущего светодиода, n_{i+1} – номер следующего светодиода, ***x*** – величина шага, лежащая в интервале [-3;+3]. Регистры PD0-PD1 отображают номер элемента ***z*** в множестве величин задержки; PD4-PD6 отображают значение ***x*** в прямом коде. Ввод числа ***y*** должен выполняться на PORTC (одновременным нажатием на несколько кнопок) при нажатой кнопке PD7. Нажатие кнопки PD7 может (а вернее должно) приводить к остановке основного функционала (вывод информации на светодиоды) до момента отпускания всех кнопок на PORTC или отпускания PD7,

	после чего основной функционал возобновиться с того же режима и состояния, но с новым значением ***y***. Кнопки PD2 (прерывание INT0) увеличивает величину шага ***x*** до +3, после чего устанавливает её равной -3 (одно нажатие – одно изменение). Кнопка PD3 (прерывание INT1) циклически изменяет время задержки ***y*** в следующем множестве {0,25 с; 0,5 с; 1 с}. Изменение величин шага и задержки должно отображаться на регистре PORTD и сохраняться во внутренний EEPROM МК в момент нажатия на кнопки PD2 и PD3, изменение логики отображения – со следующего шага «бегущего огня». Исходное положение: PORTA = ***y*** = *0x73*, величина шага ***x*** = *1*, величина задержки ***z*** извлекается из EEPROM, место и формат хранения выбирается самостоятельно.
7	Регистры PORTA–PORTB выполняют роль новогодней гирлянды. Есть следующие 3 режима работы: – на каждом регистре два состояния – вывод чисел *0xFF* и *0x00*, смена состояний с частотой ***x***; – на каждом регистре два состояния – вывод чисел *0xAA* и *0x55*, смена состояний с частотой ***x***; – на каждом регистре два состояния – вывод чисел ***y*** и ***-y****, смена состояний с частотой ***x***. Регистры PD4-PD5 отображают номер режима работы (1-3), регистр PD6 отображают номер состояния в конкретном режиме (0/1), регистры PD0-PD1 отображают номер элемента ***x*** в множестве частот. Ввод числа ***y*** должен выполняться на PORTC (одновременным нажатием на несколько кнопок) при нажатой кнопке PD7. Нажатие кнопки PD7 может (а вернее должно) приводить к остановке основного функционала (вывод информации на светодиоды) до момента отпускания всех кнопок на PORTC или отпускания PD7, после чего основной функционал возобновиться с того же режима и состояния, но с новым значением ***y***. Смена режима работы должна производиться циклически с помощью кнопки PD2 (прерывание INT0), при этом номер активного состояния должен сохраняться.

	С помощью кнопки PD3 (прерывание INT1) циклически изменяется величина ***x*** в следующем множестве {0,25 Гц; 0,5 Гц; 1 Гц}. Изменения должны отображаться на регистре PORTD и сохраняться во внутренний EEPROM МК в момент нажатия на кнопки PD2 и PD3. Исходное положение: первый режим и первое состояние (0xFF на регистрах PORTA-PORTC), величина ***x*** извлекается из EEPROM, место и формат хранения выбирается самостоятельно, ***y*** = 0x55.
8	Разрядам регистров PORTA и PORTB ставится в соответствие отрезок [0–15] (0-7 ~ PA0–PA7, 8-15 ~ PB0-PB7). На данном отрезке отображается «бегущий огонь» из нескольких горящих светодиодов (восьмиразрядное число ***y*** и ***-y****, например, если ***y***=53=*0x*35=*0b***0011.0101**, то в варианте с обратным кодом «бегущий огонь» выглядит как *0b***0011.0101.1100.1010**). Для каждого состояния активные светодиоды загорается на 0,2 с, после чего активным становится другие светодиоды. Номер следующего активного светодиода определяется по формуле: $n_{i+1} = n_i + x \ (mod\ 16)$, где n_i – номер текущего светодиода, n_{i+1} – номер следующего светодиода, ***x*** – величина шага, лежащая в интервале [-3;+3]. Разряды PD4-PD6 отображают значение ***x*** в прямом коде. Ввод числа ***y*** должен выполняться на PORTC (одновременным нажатием на несколько кнопок) при нажатой кнопке PD7. Нажатие кнопки PD7 может (а вернее должно) приводить к остановке основного функционала (вывод информации на светодиоды) до момента отпускания всех кнопок на PORTC или отпускания PD7, после чего основной функционал возобновиться с того же режима и состояния, но с новым значением ***y***. Кнопки PD2 (прерывание INT0) увеличивает величину шага ***x*** до +3, после чего устанавливает её равной -3 (одно нажатие – одно изменение на единицу). Кнопка PD3 (прерывание INT1) меняет взаимное расположение ***y*** и ***-y**** в «бегущем огне». Изменения величины шага должны отображаться на регистре PORTD и сохраняться во внутренний EEPROM МК в момент нажатия на кнопки PD2 и PD3, изменение логики отображения – со следующего шага «бегущего огня». Исходное положение:

	PORTA = ***y*** = *0x83*, величина шага ***x*** извлекается из EEPROM, место и формат хранения выбирается самостоятельно.
9	Регистры PORTA–PORTB выполняют роль новогодней гирлянды. Есть следующие 3 режима работы: – на каждом регистре два состояния – вывод чисел *0xFF* и *0x00*, смена состояний с частотой ***x***; – на каждом регистре два состояния – вывод чисел *0xAA* и *0x55*, смена состояний с частотой ***x***; – на каждом регистре два состояния – вывод чисел ***y*** и ***-y****, смена состояний с частотой ***x***. Регистры PD0-PD1 отображают номер режима работы (1-3), регистр PD4 отображают номер состояния в конкретном режиме (0/1). Ввод числа ***y*** должен выполняться на PORTC (одновременным нажатием на несколько кнопок) при нажатой кнопке PD7. Нажатие кнопки PD7 может (а вернее должно) приводить к остановке основного функционала (вывод информации на светодиоды) до момента отпускания всех кнопок на PORTC или отпускания PD7, после чего основной функционал возобновиться с того же режима и состояния, но с новым значением ***y***. Смена режима работы должна производиться циклически с помощью кнопок PD2 (прерывание INT0) и PD3 (прерывание INT1), в прямом и обратном порядке соответственно, при этом номер активного состояния должен сохраняться. Смена режима должна фиксироваться на PORTD и сохраняться во внутренний EEPROM МК в момент нажатия кнопок. Исходное положение: величина ***x*** = *2* Гц и первое состояние, ***y*** = 0x71, номер режима извлекается из EEPROM, место и формат хранения выбирается самостоятельно.
10	Разрядам регистров PORTA и PORTB ставится в соответствие отрезок [0–15] (0-7 ~ PA0–PA7, 8-15 ~ PB0-PB7). На данном отрезке отображается «бегущий огонь» из нескольких горящих светодиодов (восьмиразрядное число ***y*** и ***-y****, например, если ***y***=53=*0x*35=*0b***0011.0101**, то в варианте с обратным кодом «бегущий огонь» выглядит как *0b***0011.0101.1100.1010**). Для каждого состояния активные светодиоды загорается на 0,2 с, после чего активным становится другие светодиоды. Номер следующего активного светодиода определяется по формуле: $n_{i+1} = n_i + x$ *(mod*

	16), где n_i – номер текущего светодиода, n_{i+1} – номер следующего светодиода, ***x*** – величина шага, лежащая в интервале [-3;+3]. Если величина шага ***x*** > *0*, то PD0 = 0 и PD1 = 1, если величина шага *x* = *0*, то PD0 = 1 и PD1 = 1, если величина шага ***x*** < *0*, то PD0 = 0 и PD1 = 1; PD4-PD5 отображают значение |***x***|. Ввод числа ***y*** должен выполняться на PORTC (одновременным нажатием на несколько кнопок) при нажатой кнопке PD7. Нажатие кнопки PD7 может (а вернее должно) приводить к остановке основного функционала (вывод информации на светодиоды) до момента отпускания всех кнопок на PORTC или отпускания PD7, после чего основной функционал возобновиться с того же режима и состояния, но с новым значением ***y***. Кнопки PD2 (прерывание INT0) и PD3 (прерывание INT1) осуществляют изменение величины шага ***x*** на +1 (mod 4) и -1 (mod 4) соответственно. Изменение величины шага должно отображаться на регистре PORTD и сохраняться во внутренний EEPROM МК в момент нажатия на кнопки PD2 и PD3, изменение логики отображения – со следующего шага «бегущего огня». Исходное положение: PORTA = ***y*** = *0x71*, величина шага ***x*** извлекается из EEPROM, место и формат хранения выбирается самостоятельно.